Lasso

(L1) Regularization

What • Why • How • Graphs • Code

The regulariser that deletes useless features

Definition | Graphs | Example | Code

Data Science Journey • Part 9 →

WHAT is Lasso?

Least Absolute Shrinkage and Selection Operator

A regression that adds an L1 penalty (the sum of the ABSOLUTE values of the weights) to the loss.
It shrinks weights AND sets some exactly to zero.

Analogy

A spring clean: weak features are thrown out completely, only the useful ones stay in the model.

Two jobs

1) regularise (fight overfitting) 2) select features

Result

a SPARSE model: simpler, faster, easier to explain

Where Lasso Shines

When few features really matter

Genes vs disease

20,000 genes, 100 patients: find the handful that matter

Text classification

50,000 words: keep the few hundred that predict spam

House price model

200 columns: drop the ones that add nothing

Finance / marketing

pick the key drivers of sales or returns from many candidates

The Formula

Penalise the SUM OF ABSOLUTE weights

$$J(w) = \sum_i (y_i - \hat{y}_i)^2 \; + \; \lambda \sum_j |w_j|$$

Data term

$\Sigma\ (y - \hat{y})^2$ how well we fit the training data

L1 penalty

$\lambda\ \Sigma\ |w|$ total size of the weights (not the intercept)

λ (alpha)

bigger λ → more weights forced to exactly 0

scikit-learn's Lasso scales the data term by 1/(2n):
(1/2n)·Σ(y − ŷ)² + alpha·Σ|w|, so its alpha is not the same number as λ above.

WHY Zeros? The Pull Toward 0

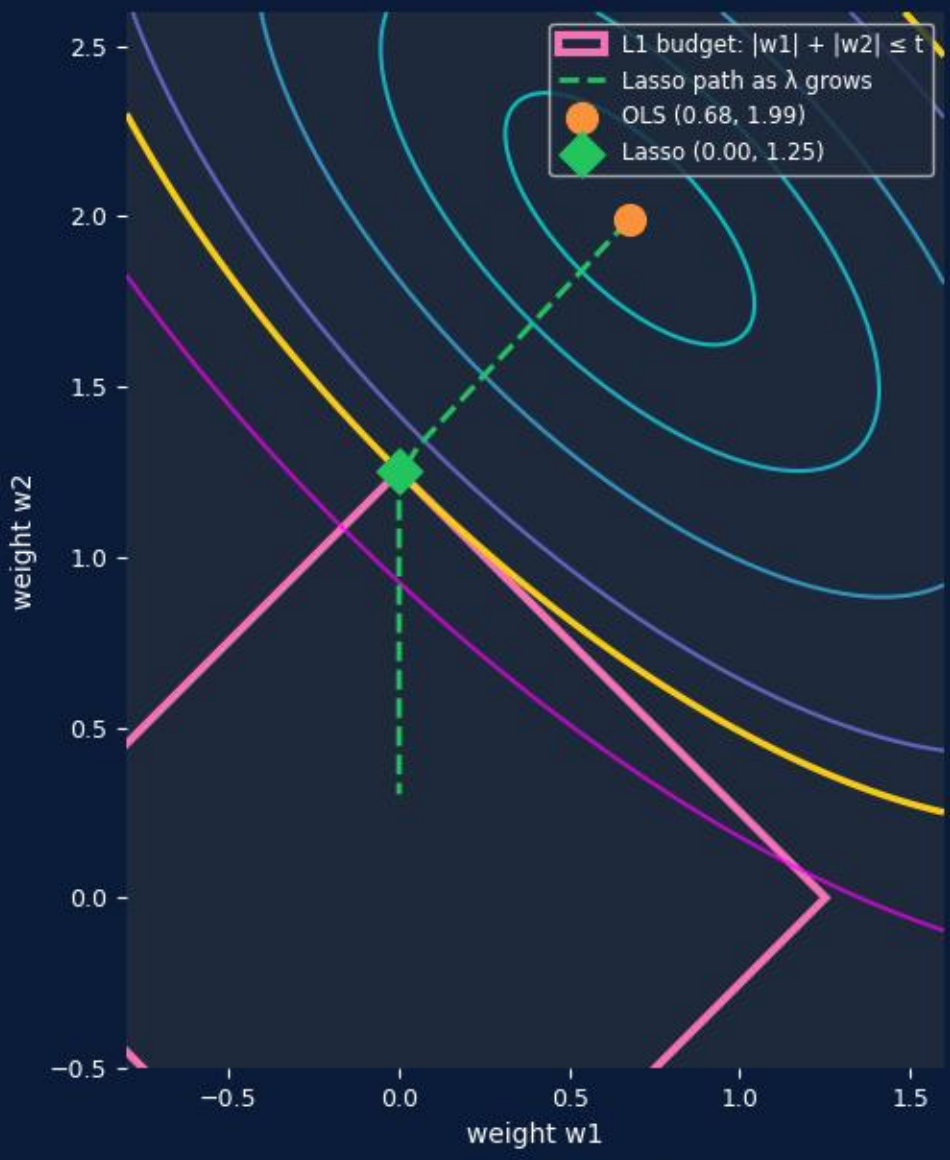
L1 pushes at a constant rate; L2 gets weaker near 0



Near zero the L2 pull vanishes, so weights get tiny but never exactly 0.
The L1 pull stays the same size, so small weights are pushed all the way to 0.

L1 Geometry: A Diamond!

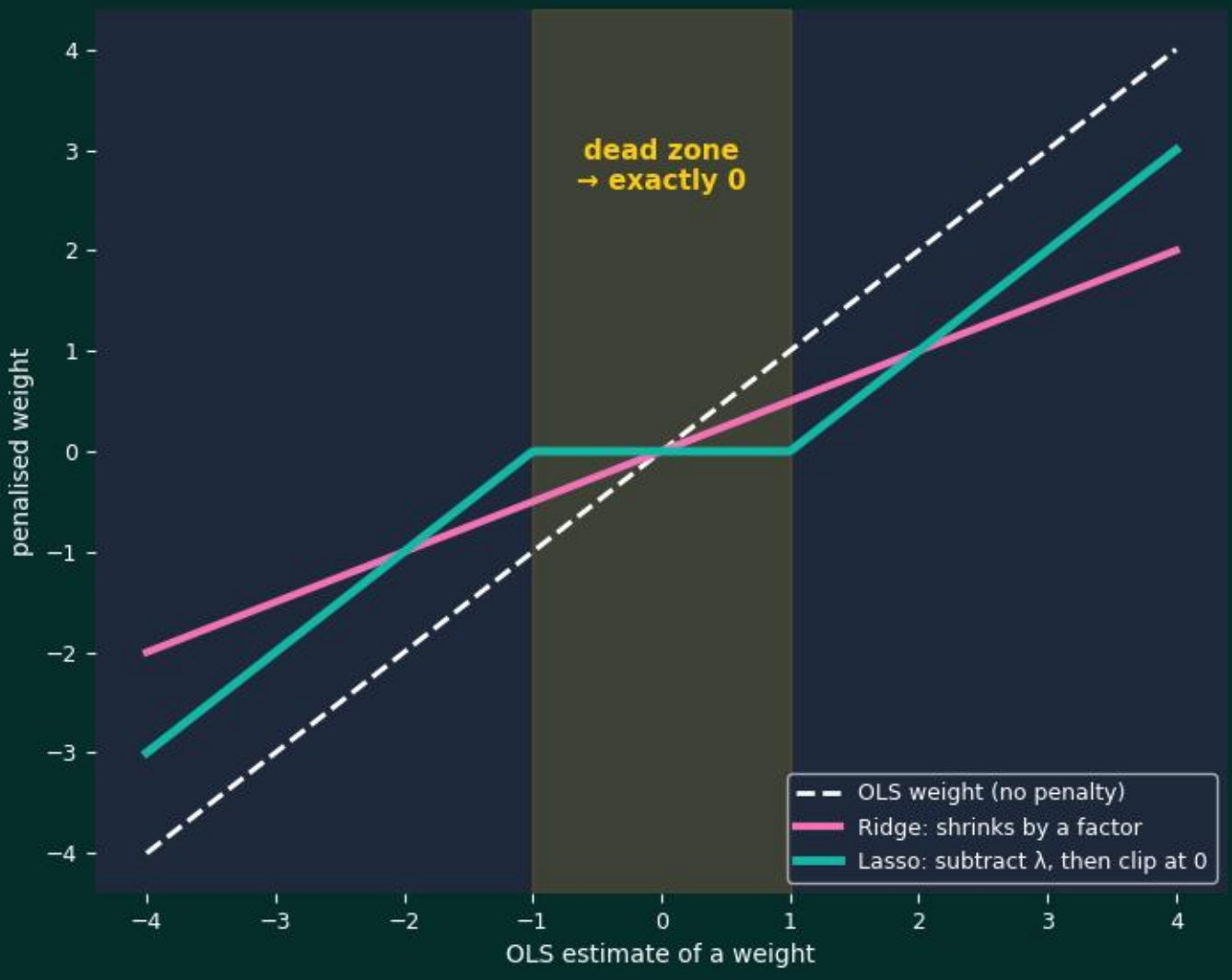
Fit contours (ellipses) vs the L1 budget (diamond)



The diamond has CORNERS on the axes. The ellipse usually touches a corner first, which makes one weight exactly 0 (here w1 = 0).

Soft-Thresholding

What Lasso does to each weight (orthogonal features)

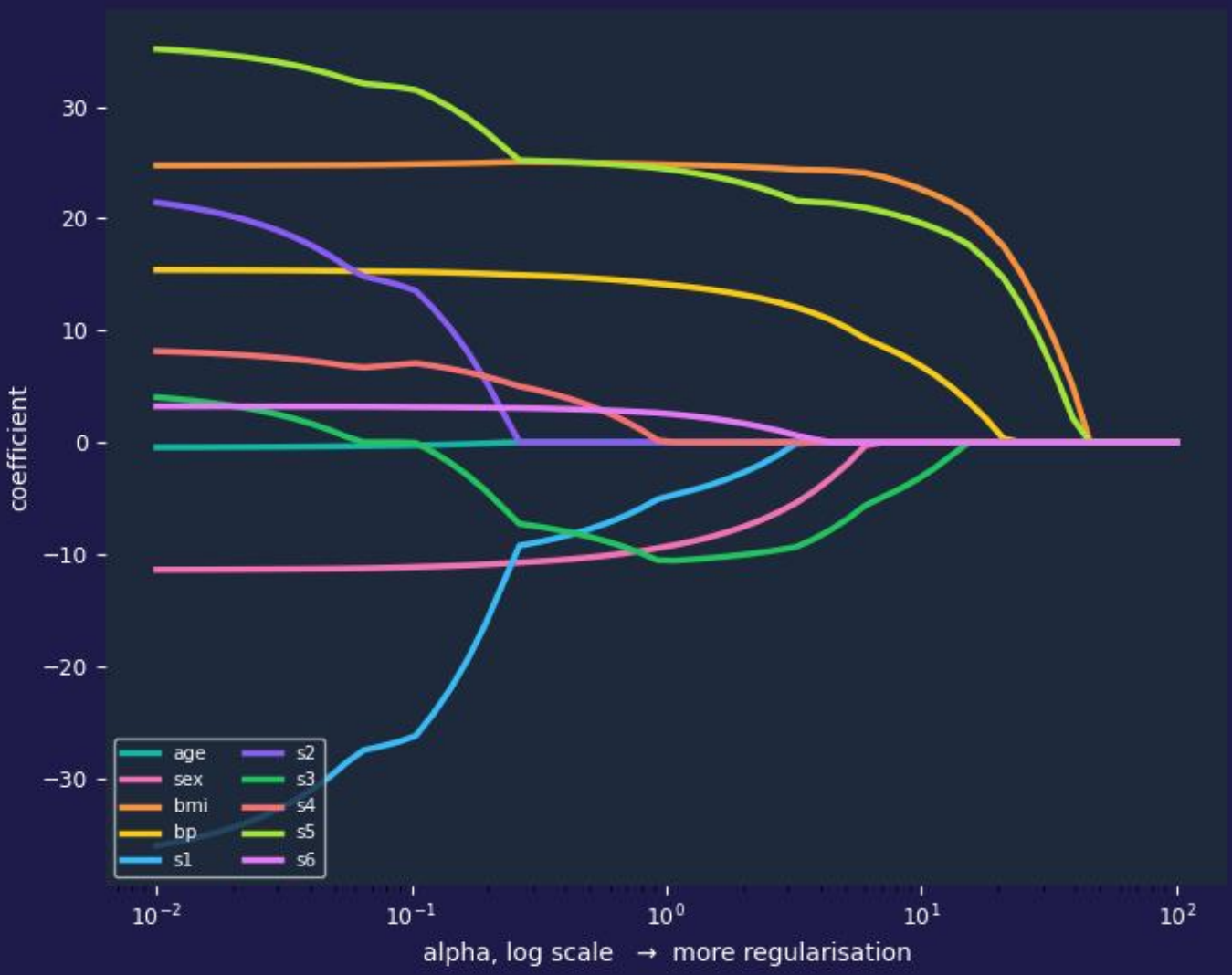


$S(z, \lambda) = \text{sign}(z) \cdot \max(|z| - \lambda, 0)$

Weights smaller than λ are deleted; bigger ones are cut by λ.

Graph: Weights Hit Zero

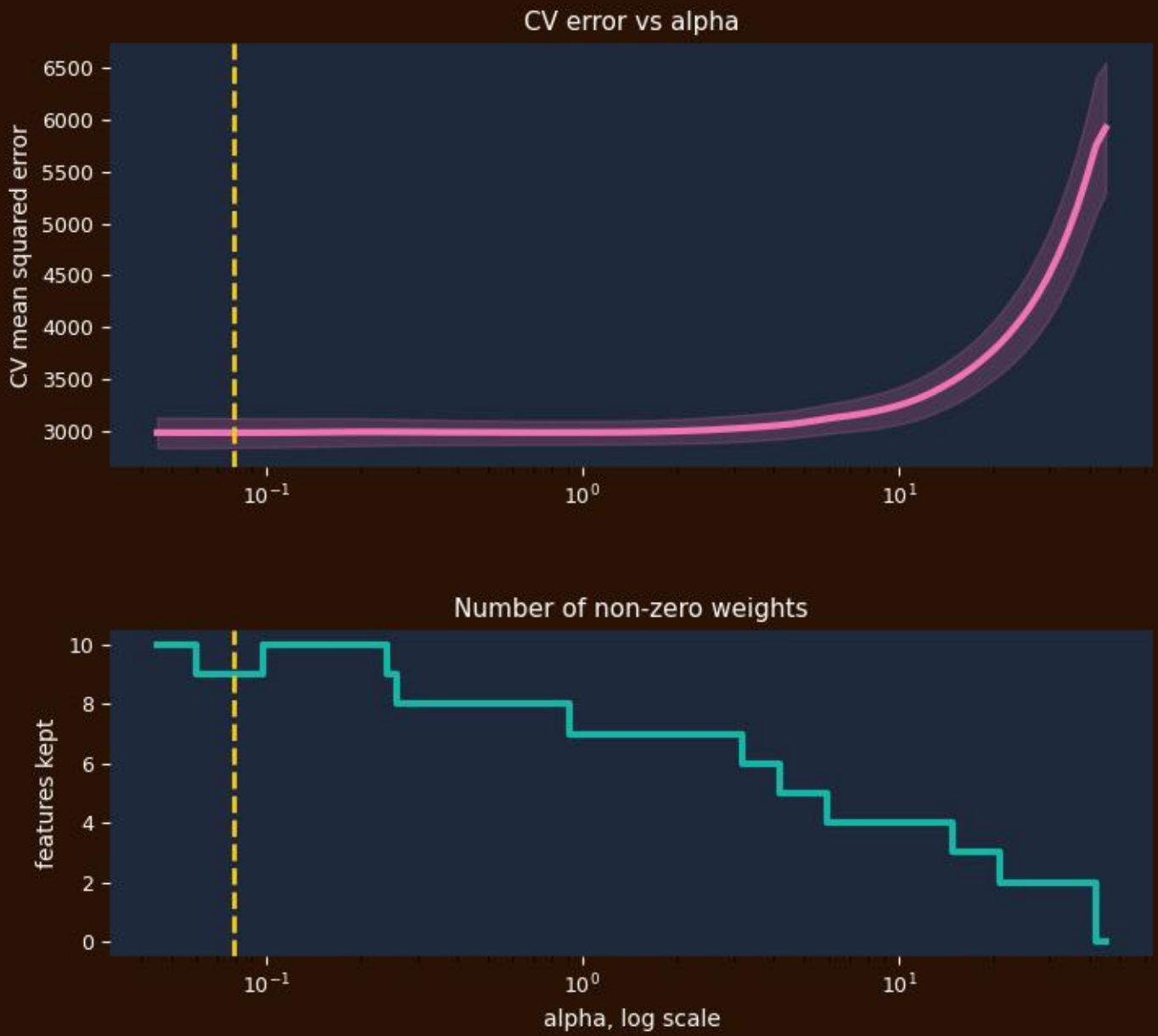
Lasso path on the Diabetes data (10 features, standardised)



As alpha grows the weights drop to exactly 0 one by one, in order of importance. The last ones standing are the strongest features.

Graph: Choosing Alpha

5-fold cross-validation (LassoCV, best alpha ≈ 0.08)



The CV minimum keeps 9 of 10 features. A larger alpha gives a simpler model.

Result: Feature Selection

Diabetes test set: OLS vs LassoCV (standardised weights)



Features kept

Lasso kept 9 of 10 (dropped: s3)

Test R^2

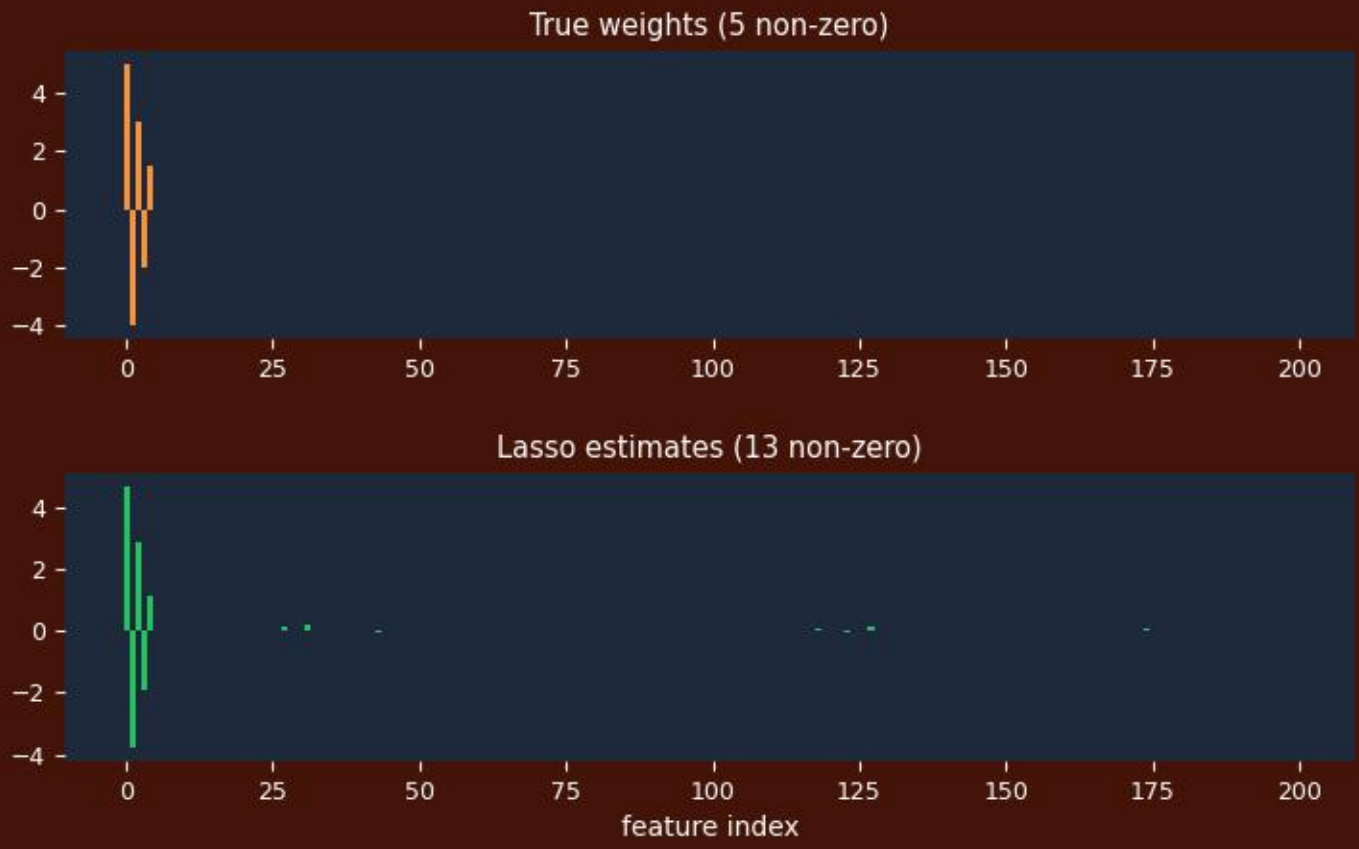
OLS 0.477 vs Lasso 0.478: same accuracy, one feature fewer

Sparser: 1-SE rule

alpha ≈ 4.12 keeps 5 of 10 features, test R^2 0.480

Power Move: p > n

200 features, 50 samples, only 5 features matter



Recovery

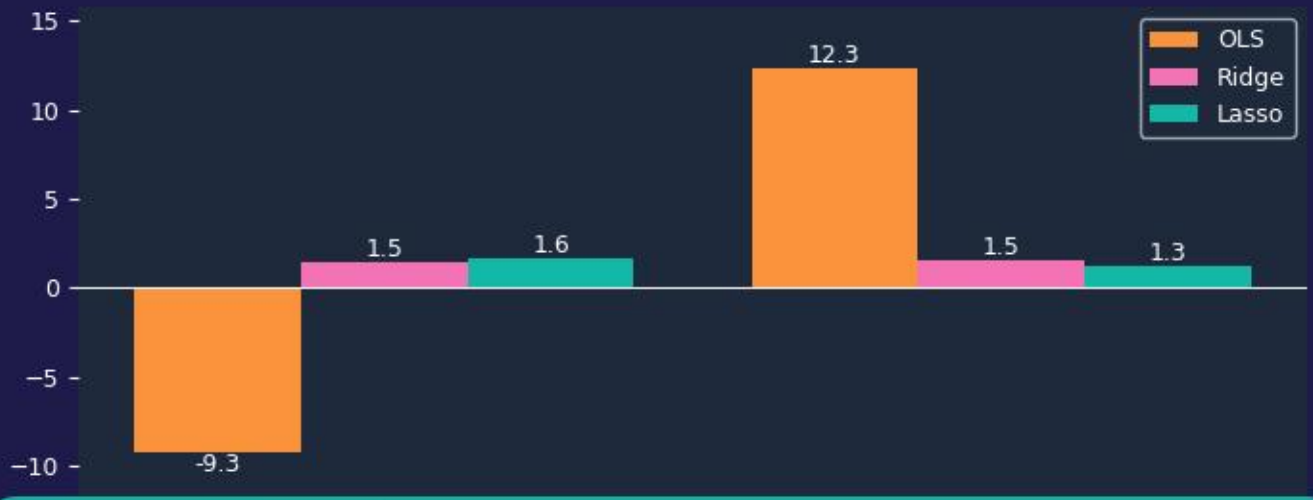
found 5 of the 5 true features; 8 false picks

Test RMSE

Lasso 0.71 vs plain OLS 6.65

Watch Out: Correlated Features

Two almost identical features (true weights 3 and 0)



Lasso: arbitrary split

total effect is kept (sum ≈ 2.9) but the split between twins
changes with every sample (std of w1 = 0.73); one twin is exactly 0 in 52% of 200 resamples

Ridge shares

splits the effect evenly: 1.5 and 1.5

Fix

Elastic Net = L1 + L2: sparse AND stable (next topic)

Lasso (L1) vs Ridge (L2)

Aspect	Lasso (L1)	Ridge (L2)
Penalty	$\lambda \Sigma \lvert w \rvert$	$\lambda \Sigma w^2$
Weights	many exactly 0	small, never exactly 0
Feature selection	YES	no
Correlated features	keeps one, drops others	shares the effect
Solution	coordinate descent (no closed form)	closed form $(X^TX + \lambda I)^{-1}X^Ty$
Best when	few features matter (sparse)	many small effects

Under the Hood

No closed form, so we use coordinate descent

$$w_j \leftarrow \frac{S(\rho_j, \lambda)}{z_j}$$

$\rho_j = \frac{1}{n} x_j^T (y - Xw + x_j w_j)$ $\quad$ $z_j = \frac{1}{n} x_j^T x_j$ $\quad$ $S(a, \lambda) = \text{sign}(a)\max(|a| - \lambda, 0)$

Why no formula?

|w| has a sharp corner at 0, so it is not differentiable there

Coordinate descent

update ONE weight at a time with soft-thresholding, cycle until it stops changing

Objective used

(1/2n)·Σ(y − ŷ)² + λ·Σ|w| (same as scikit-learn)

Code 1: Lasso in sklearn

Google Colab • path, LassoCV, selection

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import (LinearRegression, Lasso,
                                  LassoCV, lasso_path)
from sklearn.pipeline import make_pipeline
from sklearn.metrics import r2_score

d = load_diabetes(as_frame=True)
X, y = d.data, d.target
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.3, random_state=42)

# Coefficient path: weights hit zero one by one
Xs = StandardScaler().fit_transform(X)
alphas, coefs, _ = lasso_path(Xs, y - y.mean(),
                              alphas=np.logspace(-2, 2, 60))
for j, name in enumerate(X.columns):
    plt.plot(alphas, coefs[j], label=name)
plt.xscale("log"); plt.xlabel("alpha"); plt.legend()

# LassoCV chooses alpha by cross-validation
lcv = make_pipeline(StandardScaler(),
                    LassoCV(cv=5, random_state=0)).fit(Xtr, ytr)
lasso = lcv[-1]
print("best alpha:", round(lasso.alpha_, 3))
print(pd.Series(lasso.coef_, index=X.columns).round(1))
print("kept:", (lasso.coef_ != 0).sum(), "of", X.shape[1])

# Compare with plain OLS on the test set
ols = make_pipeline(StandardScaler(),
                    LinearRegression()).fit(Xtr, ytr)
print("OLS   test R2:", round(r2_score(yte, ols.predict(Xte)), 3))
print("Lasso test R2:", round(r2_score(yte, lcv.predict(Xte)), 3))
```

Code 2: Lasso From Scratch

Google Colab • soft-thresholding + coordinate descent

```
# Lasso from scratch: soft-thresholding + coordinate descent
def soft(z, t):                        # S(z, t)
    return np.sign(z) * np.maximum(np.abs(z) - t, 0)

def lasso_cd(X, y, lam, iters=300):
    n, p = X.shape; w = np.zeros(p)
    for _ in range(iters):
        for j in range(p):
            r = y - X @ w + X[:, j] * w[j]    # residual w/o feature j
            rho = X[:, j] @ r / n
            w[j] = soft(rho, lam) / (X[:, j] @ X[:, j] / n)
    return w

yc0 = y.values - y.values.mean()
w_me = lasso_cd(Xs, yc0, lam=5.0)
w_sk = Lasso(alpha=5.0, fit_intercept=False,
             tol=1e-10, max_iter=100000).fit(Xs, yc0).coef_
print("mine   :", w_me.round(2))
print("sklearn:", w_sk.round(2))
print("max difference:", np.abs(w_me - w_sk).max().round(4))
```

Code 3: p > n and Twins

Google Colab • sparse recovery, correlated features

```
# p > n: 200 features, only 5 matter, 50 samples
rng = np.random.default_rng(0)
n, p = 50, 200
Xh = rng.normal(size=(n, p))
true_w = np.zeros(p); true_w[:5] = [5, -4, 3, -2, 1.5]
yh = Xh @ true_w + rng.normal(0, 1, n)
Xnew = rng.normal(size=(500, p)); ynew = Xnew @ true_w

lc = LassoCV(cv=5, random_state=0).fit(Xh, yh)
sel = np.flatnonzero(lc.coef_)
print("selected:", len(sel), "| true found:",
      len(set(sel) & set(range(5))), "of 5")
rm = lambda m: np.sqrt(((m.predict(Xnew) - ynew) ** 2).mean())
print("test RMSE  Lasso:", round(rm(lc), 2),
      "| OLS:", round(rm(LinearRegression().fit(Xh, yh)), 2))

# Correlated twins: Lasso keeps one, drops the other
from sklearn.linear_model import Ridge
r2_ = np.random.default_rng(0)
x1 = r2_.normal(size=50); x2 = x1 + r2_.normal(0, .01, 50)
Xc = np.c_[x1, x2]; yc = 3 * x1 + r2_.normal(0, 1, 50)
print("OLS  :", LinearRegression().fit(Xc, yc).coef_.round(2))
print("Ridge:", Ridge(alpha=1).fit(Xc, yc).coef_.round(2))
print("Lasso:", Lasso(alpha=0.1).fit(Xc, yc).coef_.round(2))
```

Lasso Cheat Sheet

Use it when

many features, few matter; you want automatic feature selection

Always

scale features first • don't penalise the intercept • tune alpha with CV

Pros

sparse and interpretable, handles p > n, fast with coordinate descent

Cons

arbitrary among correlated features • can keep at most n features • biased weights

Lasso, Ridge or Elastic Net?

Comment your go-to

Repost to help a learner

Follow: next is Elastic Net

#DataScience #Lasso #Regularization #MachineLearning #Python